

Technical Design & Implementation Report
Version 1.0.0 • June 2026

Project Name	firebase_auth_app
Platform	Flutter (Android / iOS)
Dart SDK	^3.11.5
Firestore Project	fir-auth-4c440
Auth Methods	Email/Password, Google Sign-In
Report Date	June 4, 2026

1. Executive Summary

This report provides a comprehensive technical analysis of the Firebase Auth Flutter application — a fully functional mobile authentication system built with Flutter and integrated with Google's Firebase platform. The project demonstrates industry-standard patterns for user identity management in cross-platform mobile applications.

Project Highlights

- Dual authentication providers: Email/Password and Google Sign-In
- Reactive auth state management using `Firebase.authStateChanges()` stream
- Modern dark-themed UI with animated transitions and glassmorphism styling
- Clean separation of concerns via a dedicated `AuthService` layer
- Cross-platform configuration via FlutterFire CLI-generated `firebase_options.dart`
- Reusable widget library (`CustomTextField`, `GoogleSignInButton`)
- Deployed on Android with `google-services.json`; iOS configuration included

Scope of the Report

This document covers the following areas in depth:

1. Project architecture and file structure
2. Firebase project setup and configuration
3. Authentication service design and API usage
4. Screen-by-screen UI/UX breakdown
5. Widget library
6. Authentication state management
7. Dependency catalogue
8. Security considerations
9. Known limitations and improvement opportunities
10. Setup & deployment guide

2. Project Overview

2.1 Technology Stack

Framework	Flutter 3.x (Dart ^3.11.5)
Backend/Auth	Firebase Authentication (Google)
State Management	Native Flutter StreamBuilder (reactive)
Social Auth	Google Sign-In via google_sign_in ^7.2.0
Core Firebase SDK	firebase_core ^4.10.0
Auth SDK	firebase_auth ^6.1.1
Target Platforms	Android (primary), iOS (configured)

2.2 Repository Structure

The project follows the standard Flutter project layout with a deliberate separation between business logic, UI screens, and reusable widgets:

```

firebase_auth/
  lib/
    main.dart           ← Entry point; Firebase init
    firebase_options.dart ← Auto-generated platform config
    models/
      user_model.dart    ← UserModel data class
    services/
      auth_service.dart  ← All Firebase Auth calls
    screens/
      auth_wrapper.dart  ← Stream-based route guard
      login_screen.dart  ← Sign-in UI
      register_screen.dart ← Registration UI
      home_screen.dart   ← Authenticated user profile
    widgets/
      custom_textfield.dart ← Animated, focus-aware input
      google_signin_button.dart ← Custom-painted Google button
  android/
    app/google-services.json ← Android Firebase config
  pubspec.yaml              ← Dependencies & metadata
    
```

3. Firebase Configuration

3.1 Firebase Project Setup

The application is connected to a Firebase project with the identifier fir-auth-4c440. The project was provisioned with Authentication enabled for both Email/Password and Google Sign-In providers. The following platforms are registered:

Android App	com.example.firebase_auth (App ID: 1:834438858975:android:d778947aca4cd030b9416a)
iOS App	com.example.firebaseAuth (App ID: 1:834438858975:ios:a65fa482d4fb7f00b9416a)
Web App	fir-auth-4c440.firebaseio.com (for Windows/Web targets)
Sender ID	834438858975
Storage Bucket	fir-auth-4c440.firebaseio.com

3.2 FlutterFire CLI & firebase_options.dart

The project uses the FlutterFire CLI to automatically generate firebase_options.dart. This file exports a DefaultFirebaseOptions class with platform-specific FirebaseOptions constants, covering Android, iOS, macOS, Windows, and Web. The app calls this during initialization:

```
await Firebase.initializeApp(
  options: DefaultFirebaseOptions.currentPlatform,
);
```

The currentPlatform getter uses kIsWeb and defaultTargetPlatform to select the correct configuration at runtime, ensuring zero-configuration platform switching.

3.3 Android Configuration

The google-services.json file is placed at android/app/google-services.json and is consumed by the com.google.gms:google-services Gradle plugin. This file contains the OAuth client configuration required for Google Sign-In on Android, including the client_id and certificate hashes registered in the Firebase Console.

- Minimum SDK: configured per Flutter defaults (typically API 21+)
- The AndroidManifest.xml includes Internet permission for Firebase network calls
- Kotlin-based MainActivity at com.example.firebase_auth.MainActivity

4. Architecture & Authentication Service

4.1 Architectural Overview

The application follows a layered architecture pattern appropriate for small-to-medium Flutter apps:

- Presentation Layer: Screens and Widgets (lib/screens/, lib/widgets/)
- Service Layer: AuthService class (lib/services/auth_service.dart)
- Data Model Layer: UserModel (lib/models/user_model.dart)
- Infrastructure Layer: Firebase SDK + google_sign_in package

The AuthWrapper screen acts as a reactive route guard sitting between the service and presentation layers, routing users based on the live Firebase auth state stream.

4.2 AuthService — Method Reference

AuthService is a plain Dart class (not a singleton or provider) that wraps the FirebaseAuth.instance and GoogleSignIn.instance APIs. It is instantiated directly in each screen (AuthService()). The service exposes the following interface:

Member	Return Type	Description
<code>authStateChanges (getter)</code>	<code>Stream<User?></code>	Emits User object on sign-in, null on sign-out. Used by AuthWrapper.
<code>currentUser (getter)</code>	<code>User?</code>	Synchronous access to the currently authenticated Firebase user.
<code>register(email, password)</code>	<code>Future<UserCredential></code>	Creates a new account via createUserWithEmailAndPassword().
<code>login(email, password)</code>	<code>Future<UserCredential></code>	Signs in an existing user via signInWithEmailAndPassword().
<code>logout()</code>	<code>Future<void></code>	Signs out from both Google Sign-In and Firebase Auth.
<code>signInWithGoogle()</code>	<code>Future<UserCredential></code>	Initializes Google Sign-In, authenticates, creates a Firebase credential from the ID token, and signs in.

4.3 Google Sign-In Flow (Detailed)

The Google Sign-In implementation uses the updated google_sign_in ^7.2.0 API, which replaces the older GoogleSignIn() constructor pattern with a static instance approach:

```

// Step 1: Initialize the Google Sign-In instance
await GoogleSignIn.instance.initialize();

// Step 2: Trigger the OAuth consent screen
final GoogleSignInAccount googleUser =
    await GoogleSignIn.instance.authenticate();

// Step 3: Retrieve ID token from the authenticated account
final GoogleSignInAuthentication googleAuth =
    googleUser.authentication;

// Step 4: Create a Firebase credential from the Google ID token
final credential = GoogleAuthProvider.credential(
    idToken: googleAuth.idToken,
);

// Step 5: Sign in to Firebase with the Google credential
return await _auth.signInWithCredential(credential);

```

Note: The access token is omitted from the credential, which is acceptable when only Firebase Auth is the target. If server-side Google API access is needed, the accessToken should also be included.

5. Screen-by-Screen Analysis

5.1 AuthWrapper

AuthWrapper is the root widget of the MaterialApp and serves as the reactive navigation hub. It subscribes to Firebase's `authStateChanges()` stream using a `StreamBuilder`:

```
StreamBuilder<User?>(
  stream: FirebaseAuth.instance.authStateChanges(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting)
      return LoadingScaffold;
    if (snapshot.hasData) return HomeScreen();
    return LoginScreen();
  },
)
```

- `ConnectionState.waiting`: Displays a centered `CircularProgressIndicator` while Firebase resolves the persisted auth state from secure storage on first launch.
- `snapshot.hasData` (User present): Routes to `HomeScreen`.
- `snapshot.hasData == false` (null User): Routes to `LoginScreen`.

This pattern means navigation is entirely driven by the auth stream — no manual `Navigator.pushNamed()` calls are required after sign-in or sign-out; the stream emission triggers a rebuild that swaps the widget tree automatically.

5.2 LoginScreen

Purpose

The login screen is the default unauthenticated landing page. It supports both email/password sign-in and Google Sign-In via a single unified screen.

State & Controllers

- `emailController`, `passwordController` — `TextEditingController` instances for the two input fields.
- `_isLoading` (bool) — Disables buttons and shows a progress indicator during async operations.
- `_animController` (`AnimationController`) — 900ms entry animation.
- `_fadeAnim` (`FadeTransition`) — Fades content from 0→1 opacity.
- `_slideAnim` (`SlideTransition`) — Slides content up 8% of its height on entry.

UI Composition

- Dark background (#0A0E1A) with two decorative radial-gradient circular blobs (teal top-right, blue bottom-left).
- Gradient logo icon (teal→blue lightning bolt in a rounded rectangle).
- Large bold heading: "Welcome\nback."

- CustomTextField for email (email keyboard type) and password (obscured).
- "Forgot password?" text button (currently a no-op placeholder).
- Gradient Sign In button (teal→blue with blue glow shadow).
- "or continue with" divider row.
- GoogleSignInButton widget.
- Navigation link to RegisterScreen.

Error Handling

All auth exceptions are caught and surfaced via a floating SnackBar with a red (#E53935) background. The error message string is displayed directly from the caught exception, which for `firebase_auth` surfaces as `FirebaseAuthException.message`.

5.3 RegisterScreen

Purpose

The registration screen allows new users to create accounts using email and password. It is reached from a navigation push from LoginScreen and pops back to LoginScreen on success.

State & Controllers

- `emailController`, `passwordController`, `confirmController` — Three text input controllers.
- `_isLoading` — Same loading guard pattern as LoginScreen.
- Same 900ms fade + slide animation stack as LoginScreen.

Client-Side Validation

Before calling `AuthService.register()`, the screen checks that `passwordController.text == confirmController.text`. If they do not match, it shows an error snackbar immediately without making any network call. A password hint ("Use 8+ characters with a mix of letters & numbers") is displayed as a static text hint below the fields.

UI Extras

- A custom back button (ghost-styled with border) replaces the system back arrow.
- A three-dot step indicator row (one active wide dot, two inactive dots) implies multi-step onboarding, suggesting extensibility for profile or preference steps.
- On success, `Navigator.pop(context)` returns to the Login screen; the `AuthWrapper` stream automatically detects the new user and routes to HomeScreen.

5.4 HomeScreen

Purpose

HomeScreen is the post-authentication destination. It displays a full user profile with account metadata, auth method indicator, and settings placeholders.

User Data Retrieval

User information is retrieved synchronously from `FirebaseAuth.instance.currentUser` at build time:

- `displayName` — falls back to "Firebase User" if null (common for email-only accounts).
- `email` — falls back to "No Email".
- `photoURL` — Used as the `CircleAvatar` `backgroundImage`; if null, initials are shown instead.
- `uid` — Displayed truncated (first 12 characters + "...") in a monospace font.

Profile Avatar Logic

The avatar is a 110x110 `CircleAvatar` inside a 3px gradient-bordered `Container`. If the user has a `photoURL` (e.g., from Google Sign-In), it is loaded via `NetworkImage`. Otherwise, initials are derived by splitting the display name on spaces and joining the first letter of each word (max 2 initials), rendered in 34pt bold white text.

Stat Cards

Two side-by-side stat cards show:

- Account: Always "Active" with a teal verified icon.
- Auth: "Google" if `photoURL` is non-null, otherwise "Email" — a heuristic that correctly identifies the sign-in provider for typical use cases.

Settings Rows (Placeholders)

Three action rows (Edit Profile, Notifications, Privacy & Security) are wired to empty `onTap` callbacks, indicating planned features not yet implemented.

Sign Out

An outlined red button calls `AuthService().logout()`, which awaits both `GoogleSignIn.instance.signOut()` and `FirebaseAuth.signOut()`. Signing out causes the `authStateChanges` stream to emit null, and `AuthWrapper` immediately routes back to `LoginScreen`.

6. Widget Library

6.1 CustomTextField

CustomTextField is a StatefulWidget that wraps Flutter's standard TextField with focus-aware animated styling. Key features:

Props

controller	TextEditingController (required) — Text input controller
hintText	String (required) — Placeholder text
icon	IconData (required) — Leading prefix icon
obscureText	bool (default false) — Enables password masking
keyboardType	TextInputType? — Optional keyboard type (e.g. emailAddress)

Animated Focus Behaviour

The widget uses a Focus wrapper to detect focus changes. When focused:

- The container border color transitions from white/8% to teal/60% (animated via AnimatedContainer at 200ms).
- The border width increases from 1px to 1.5px.
- A teal glow box shadow appears beneath the container.
- The prefix icon color transitions from white/40% to teal/80%.

Password Visibility Toggle

When obscureText is true, a suffix IconButton renders an eye icon (visibility_off when hidden, visibility when shown). Tapping toggles the internal _obscured bool, which is passed to TextField.obscureText.

6.2 GoogleSignInButton

GoogleSignInButton is a custom-styled button that renders the official Google "G" logo using Flutter's Canvas API, avoiding any external SVG or image asset dependency.

Press Animation

The button maintains a _isPressed bool toggled by GestureDetector.onTapDown/onTapUp/onTapCancel. The AnimatedContainer transitions background opacity from 4% to 8% white on press.

_GoogleLogoPainter (CustomPainter)

The Google logo is drawn procedurally:

11. A white circle background is drawn.

12. Four 90° pie slices are drawn in the canonical Google colors: red (#EA4335), blue (#4285F4), green (#34A853), yellow (#FBBC05).
13. An inner white circle (62% of radius) creates the donut/ring effect.
14. A blue horizontal rectangle covers the right half at mid-height, forming the crossbar of the "G".
15. A white arc re-covers the left half of the inner circle to clean up the bar overlap.

This approach ensures the button renders identically at any device pixel ratio without rasterisation artifacts.

Loading State

When `isLoading` is true, the Google logo and label are replaced with a small `CircularProgressIndicator` (20x20, white, `strokeWidth` 2). The `InkWell` `onTap` is set to null while loading to prevent double-taps.

7. Data Model

7.1 UserModel

The app defines a lightweight UserModel class in lib/models/user_model.dart:

```
class UserModel {  
  final String uid;  
  final String? name;  
  final String? email;  
  final String? photoUrl;  
  
  UserModel({required this.uid, this.name, this.email, this.photoUrl});  
}
```

This model mirrors a subset of the Firebase User object. In the current implementation, screens use FirebaseAuth.instance.currentUser directly rather than mapping to UserModel — meaning UserModel is currently defined but not yet consumed. It is well-positioned for future use in a state management solution (e.g., Provider or Riverpod) where a typed domain model is preferable to passing the raw Firebase User object through the widget tree.

8. Dependency Catalogue

8.1 Production Dependencies

Package	Version	Purpose
<code>flutter</code>	SDK	Core Flutter framework
<code>firebase_core</code>	^4.10.0	Firebase SDK initialization; required by all Firebase packages
<code>firebase_auth</code>	^6.1.1	Firebase Authentication (email/password, credential sign-in)
<code>google_sign_in</code>	^7.2.0	Google OAuth 2.0 sign-in flow; provides ID token for Firebase
<code>cupertino_icons</code>	^1.0.8	iOS-style icon assets for CupertinoIcons class

8.2 Dev Dependencies

Package	Version	Purpose
<code>flutter_test</code>	SDK	Flutter testing framework (unit, widget, integration tests)
<code>flutter_lints</code>	^6.0.0	Recommended lint rules enforced via <code>analysis_options.yaml</code>

9. Security Considerations

9.1 Credential Handling

- Passwords are never logged or stored in application state; they are passed directly to the Firebase SDK and discarded after the API call.
- The passwordController is disposed in the widget's dispose() method, freeing the memory immediately when the screen is removed from the tree.
- Google ID tokens are ephemeral — they are created in signInWithGoogle(), passed to FirebaseAuthProvider.credential(), and not retained.

9.2 firebase_options.dart

The firebase_options.dart file contains API keys and app identifiers in plaintext. It is important to understand the security model:

- Firebase API keys for client-side SDKs are not secret — they identify the Firebase project but do not grant administrative access.
- Access to Firebase resources is controlled by Firebase Security Rules, not by the API key.
- However, the file should still be included in .gitignore if the repository is public, to reduce the risk of quota abuse.
- The iOS client ID embedded in firebase_options.dart is used for Google Sign-In OAuth flow and should be kept out of public repositories.

9.3 Auth State Persistence

Firebase Auth for Flutter persists the user session to platform-native secure storage by default (Keychain on iOS, EncryptedSharedPreferences on Android). This means users remain authenticated across app restarts without re-entering credentials. The AuthWrapper handles the loading state during session restoration.

9.4 Error Message Exposure

The current implementation displays raw exception messages in SnackBars (e.toString()). FirebaseAuthException messages are generally user-friendly but may reveal information about whether an email is registered. For production hardening, consider:

- Mapping FirebaseAuthException.code values to custom, localized error strings.
- Avoiding messages like "No user found for this email" that confirm account existence (account enumeration vulnerability).

10. Limitations & Recommended Improvements

10.1 Current Limitations

No Form Validation

The login and register screens do not validate inputs before making API calls. An empty email field will result in a Firebase error rather than an inline field-level error message. Recommended fix: use a Form widget with TextFormField and a GlobalKey<FormState> to perform client-side validation before any async call.

AuthService Instantiation Pattern

AuthService() is instantiated with new in each screen callback. While functional, this creates multiple instances. The idiomatic approach in Flutter is to use a package like provider, riverpod, or get_it to inject a single AuthService instance, making the service easier to mock in tests.

UserModel Not Integrated

The UserModel class is defined but never used — screens access FirebaseAuth.instance.currentUser directly. Integrating UserModel into a ChangeNotifier or StateNotifier would decouple the UI from Firebase types and enable easier unit testing.

No Password Reset Implementation

The "Forgot password?" button has an empty onPressed callback. Firebase Auth provides sendPasswordResetEmail() which should be wired to this button.

Google Auth Heuristic on HomeScreen

The "Auth: Google / Email" stat card determines the provider by checking photoURL != null. This is not reliable — a user could upload a photo after email/password registration. The correct approach is to check user.providerData[0].providerId.

No Email Verification

After email/password registration, no verification email is sent. Firebase Auth provides sendEmailVerification() on the User object. Without this, users can register with any email address.

Settings Rows Are Placeholders

Edit Profile, Notifications, and Privacy & Security action rows all have empty onTap callbacks. These features are not implemented.

10.2 Recommended Next Steps

Form Validation	Add GlobalKey<FormState> + validator callbacks to all TextFormFields
Password Reset	Implement sendPasswordResetEmail() on the "Forgot password?" tap
Email Verification	Call user.sendEmailVerification() after registration

State Management	Adopt Riverpod or Provider to inject a shared AuthService
UserModel Integration	Map Firebase User to UserModel; expose via StateNotifier
Provider ID Fix	Use user.providerData[0].providerId instead of photoURL heuristic
Unit Tests	Add auth_service_test.dart with mocked FirebaseAuth
Widget Tests	Add login_screen_test.dart and register_screen_test.dart
Deep Links (Android)	Add intent-filter for OAuth redirect to AndroidManifest.xml
Apple Sign-In	Add sign_in_with_apple package for App Store compliance on iOS

11. Setup & Deployment Guide

11.1 Prerequisites

- Flutter SDK ^3.11.5 installed and on PATH
- Dart SDK ^3.11.5 (bundled with Flutter)
- Android Studio or VS Code with Flutter extension
- Firebase account with a project created
- Google Cloud Console: OAuth 2.0 client ID created for the Android package name

11.2 Firebase Project Setup

16. Go to <https://console.firebase.google.com> and create a new project.
17. Navigate to Authentication > Sign-in method and enable Email/Password and Google.
18. Add an Android app with package name com.example.firebase_auth.
19. Download google-services.json and place it at android/app/google-services.json.
20. (Optional) Add an iOS app with bundle ID com.example.firebaseAuth and download GoogleService-Info.plist.

11.3 FlutterFire CLI Setup

```
# Install FlutterFire CLI
dart pub global activate flutterfire_cli

# Configure the project (run from the project root)
flutterfire configure --project=fir-auth-4c440
```

This regenerates `firebase_options.dart` for all registered platforms. Re-run whenever you add a new platform or rotate keys.

11.4 Install Dependencies & Run

```
cd firebase_auth
flutter pub get
flutter run
```

For a release build:

```
flutter build apk --release          # Android APK
flutter build appbundle --release    # Android AAB (Play Store)
flutter build ios --release          # iOS (requires Xcode on macOS)
```

11.5 Google Sign-In — Android Debug Setup

Google Sign-In requires the SHA-1 fingerprint of your debug keystore to be registered in the Firebase Console under Android App Settings:

```
keytool -list -v -keystore ~/.android/debug.keystore \  
-alias androiddebugkey \  
-storepass android -keypass android
```

Copy the SHA-1 value and add it in Firebase Console > Project Settings > Your apps > Android app > Add fingerprint.

12. Conclusion

The Firebase Auth Flutter application is a well-structured, visually polished foundation for mobile authentication. It successfully demonstrates the core Firebase Auth API surface — including both email/password and Google OAuth flows — within a clean architectural separation of services, screens, and reusable widgets.

The use of Firebase's `authStateChanges()` stream as the single source of navigation truth is a best-practice pattern that eliminates an entire class of navigation bugs. The animated entry transitions and focus-aware input styling elevate the UI above typical tutorial-level implementations.

The primary areas for growth are: wiring up form validation and error mapping for production-grade UX; integrating a dependency injection pattern for testability; implementing the stub features (password reset, email verification, edit profile); and adding automated tests for the service and screen layers.

Overall, this codebase represents a solid, deployable starting point and demonstrates sound Flutter and Firebase engineering fundamentals.

End of Report • Firebase Auth Flutter App • v1.0.0 • June 2026